



Projektbericht

ZAIA - Zotero AI Assistant

Modul:	Wirtschaftsinformatik Projekt
Semester:	Sommersemester 2026
Betreuer:	Till Schirrmeister
Projektteam:	Frank Baum, Vincent Korsch, Nicolas Medda Timo Rausch, Henrik Schneider
Abgabedatum:	25.07.2026

1 Projektüberblick

1.1 Ausgangssituation und Problemstellung

Zotero unterstützt die Sammlung, Strukturierung und Zitation wissenschaftlicher Literatur. Für inhaltliche Fragen, Vergleiche oder Zusammenfassungen müssen Nutzer:innen ihre Quellen jedoch häufig in ein separates KI-Werkzeug übertragen. Dadurch entstehen Medienbrüche: Dokumente werden mehrfach ausgewählt oder hochgeladen, der Bezug zur bestehenden Bibliotheksstruktur geht verloren und bei externen Diensten ist nicht immer transparent, welche Inhalte den lokalen Arbeitsplatz verlassen.

ZAIA adressiert dieses Problem als direkt in Zotero eingebetteter Forschungsassistent. Das Plugin verbindet den Chat mit den bereits vorhandenen Einträgen, PDF-Dateien, Metadaten und Gruppenbibliotheken. Nutzer:innen können dabei zwischen einem institutionellen Cloud-Zugang über KISSKI/Academic Cloud und einer lokalen Verarbeitung mit Ollama wählen. Damit soll ein durchgängiger Arbeitsablauf von der Literaturverwaltung bis zur kontextbezogenen KI-Anfrage entstehen.

1.2 Zielbild und Anforderungen

Das Projektziel war ein installierbarer, alltagstauglicher Prototyp, der folgende Kernanforderungen erfüllt:

- native Bedienung in Zotero über Sidebar, Pop-out und Tastenkürzel,
- austauschbare lokale und cloudbasierte Sprachmodelle,
- gezielte Auswahl von Paper-Kontext sowie bibliotheksweite Suche,
- lokale semantische Suche mit kontrollierbarer Indexierung,
- persistente, verwaltbare Chatverläufe,
- Unterstützung persönlicher und gemeinsamer Zotero-Bibliotheken sowie
- ein geführter Setup-Prozess für die technisch anspruchsvolle lokale KI.

Das Ergebnis liegt als Version 1.0.0 vor und setzt Zotero ab Version 9.0.4 voraus. Der Quellcode ist als AGPL-3.0-or-later deklariert. Neben dem Add-on gehören eine Projektwebsite, ein öffentlicher GitHub-Auftritt und eine deutschsprachige Bedienoberfläche zum erreichten Projektstand.

1.3 Projektabgrenzung

Der Schwerpunkt lag auf einer Desktop-Erweiterung für Zotero, nicht auf einer eigenständigen Web- oder Mobilanwendung. Das automatische Installieren und Starten von Ollama ist auf Windows und macOS ausgelegt; entfernte Ollama-Endpunkte werden zwar angesprochen, aber nicht durch ZAIA verwaltet. Nicht Gegenstand des bisherigen Stands sind eine fachliche Garantie für KI-Antworten, eine automatische Prüfung erzeugter Literaturangaben oder eine belastbare Messung der Zeitersparnis im realen Forschungsalltag.

2 Key-Deliveries und Zielerreichung

2.1 Ergebnisübersicht

Deliverable	Erreichter Stand	Status
Zotero-Integration	Sidebar, Toolbar-Zugang und separates Pop-out-Fenster im Zotero-Add-on-Kontext	Umgesetzt
Lokale und Cloud-KI	Umschaltbare Provider für KISSKI/Academic Cloud und Ollama einschließlich Modellauswahl	Umgesetzt
Kontext und RAG	Manuelle Paper-Auswahl, Metadaten-Routing, PDF-Chunking, Hybrid- bzw. Keyword-Suche	Umgesetzt
Index-Management	Bibliothekübergreifender Index, Statusanzeige, Filter, Einzel-/Mehrfachaktionen und Abbruch	Umgesetzt
Group Libraries	Erkennung persönlicher und gemeinsamer Bibliotheken samt bibliotheksbezogener Metadaten	Umgesetzt
Chatverlauf	Lokale SQLite-Speicherung, Titel, Favoriten, Löschen und optionale Bereinigung alter Chats	Umgesetzt
Setup und Betrieb	Readiness-Anzeige, signaturgeprüfte Ollama-Installation, Modell-Download und Prozessverwaltung	Umgesetzt
Qualitätsnachweis	Automatisierte Unit- und Integrationstests; reproduzierbarer Produktions-Build	Umgesetzt
Praxisevaluation	Systematische Nutzungsstudie und quantitative Nutzenmessung	Teilweise

2.2 Nutzeroberfläche und Arbeitsablauf

Die zentrale Oberfläche ist eine ZAIA-Seitenleiste innerhalb von Zotero. Sie vereint Chatverlauf, Prompt-Eingabe, Provider- und Modellauswahl sowie die Kontextsteuerung. Ausgewählte Zotero-Einträge werden automatisch als möglicher Paper-Kontext erkannt; weitere Paper lassen sich über eine Bibliothekssuche manuell hinzufügen und auch wieder entfernen. Eine Badge zeigt die Zahl der aktiven Kontextquellen. Antworten werden als strukturiertes Markdown mit Überschriften, Listen, Tabellen, Code- und Mathematikblöcken dargestellt und können kopiert werden. Laufende Anfragen lassen sich abbrechen.

Für längere Arbeitsphasen kann dieselbe Chatoberfläche in einem eigenen Fenster geöffnet werden. Sechs plattformgerechte Tastenkürzel decken häufige Aktionen wie Öffnen, neuen Chat, Pop-out, Favorisieren, Kontext und Modellauswahl ab, ohne bekannte Zotero-Standardkürzel zu überschreiben. Die Oberfläche enthält außerdem eine Projekt- und Teamansicht mit Links zu Website, GitHub und Instagram.

2.3 Flexible KI-Anbindung und geführtes Setup

Eine Provider-Abstraktion trennt die Fachlogik von den konkreten KI-Endpunkten. Im Cloud-Modus nutzt ZAIA eine OpenAI-kompatible KISSKI-Schnittstelle und lädt die dort verfügbaren

Modelle dynamisch. Im lokalen Modus spricht das Plugin die Ollama-API an; als schlankes Standardmodell ist `qwen2.5:3b` vorkonfiguriert. Weitere Chatmodelle können in einem eigenen Modellfenster heruntergeladen, ausgewählt und gelöscht werden.

Der Setup-Status wird nicht als einzelner Fehler, sondern als Folge von Meilensteinen dargestellt: Cloud-Verbindung, Ollama-Installation, Ollama-Dienst, lokales Chatmodell und Embedding-Modell. Fehlt Ollama, startet ZAIA unter Windows beziehungsweise macOS einen Helfer, der das offizielle Installationsartefakt lädt und dessen Plattformsignatur prüft. Ist Ollama zwar installiert, aber nicht erreichbar, kann ZAIA `ollama serve` im Hintergrund starten. Beim Beenden des Add-ons wird nur ein Prozess beendet, den ZAIA selbst gestartet hat.

2.4 Kontextauswahl, Indexierung und Retrieval

ZAIA extrahiert Text aus PDF-Anhängen und zerlegt ihn standardmäßig in Abschnitte von ungefähr 512 Tokens mit 100 Tokens Überlappung. Abstrakte werden als eigene Quellen berücksichtigt. Für die semantische Suche erzeugt das lokale Embedding-Modell `bge-m3:latest` Vektoren mit 1024 Dimensionen. Die Dokumente und Vektoren werden in einer lokalen Orama-Datenbank gespeichert. Je nach Verfügbarkeit kombiniert die Suche Vektorähnlichkeit und Volltextsuche oder fällt kontrolliert auf eine stichwortbasierte Suche zurück.

Ein vorgeschalteter Kontext-Router unterscheidet unter anderem zwischen allgemeinen Fragen, reinen Metadatenfragen, einem einzelnen Paper, gefilterten Papern und der gesamten Bibliothek. Dadurch muss nicht für jede Anfrage der gesamte Bestand an das Chatmodell übergeben werden. Der Index-Manager macht den Zustand transparent: Er zeigt indexierte und nicht indexierte Paper, bietet Bibliotheks-, Typ-, Jahres- und Textfilter und ermöglicht Neuaufbau, Leeren, gezielte Indexierung sowie Abbruch laufender Vorgänge.

2.5 Chatverwaltung und Produktivitätsfunktionen

Chats und Nachrichten werden in einer lokalen SQLite-Datenbank im Zotero-Datenverzeichnis abgelegt. Gespeichert werden neben Inhalt und Reihenfolge auch Zeitstempel, Bibliotheks-/Item-Bezug, Favoritenstatus und — sofern vom Provider geliefert — Tokenzahlen. Der erste Prompt kann zur automatischen Benennung eines Chats verwendet werden. Nicht favorisierte Chats lassen sich optional nach 14 Tagen automatisch entfernen; Favoriten bleiben erhalten.

In Summe wurden die funktionalen Kernziele erreicht. Teilweise offen bleibt die Validierung außerhalb des Entwicklungsteams: Der vorhandene Testbestand prüft zentrale Softwarepfade, ersetzt aber keine langfristige Feldstudie mit Studierenden und Forschenden.

3 Softwarearchitektur

3.1 Architekturüberblick

ZAIA läuft vollständig im Zotero-Add-on-Kontext. Die Architektur trennt Darstellung, Ablaufsteuerung, fachliche Kontextaufbereitung, Provider und lokale Persistenz. Externe Kommunikation findet nur mit dem gewählten Cloud-LLM oder dem konfigurierten Ollama-Endpunkt statt.

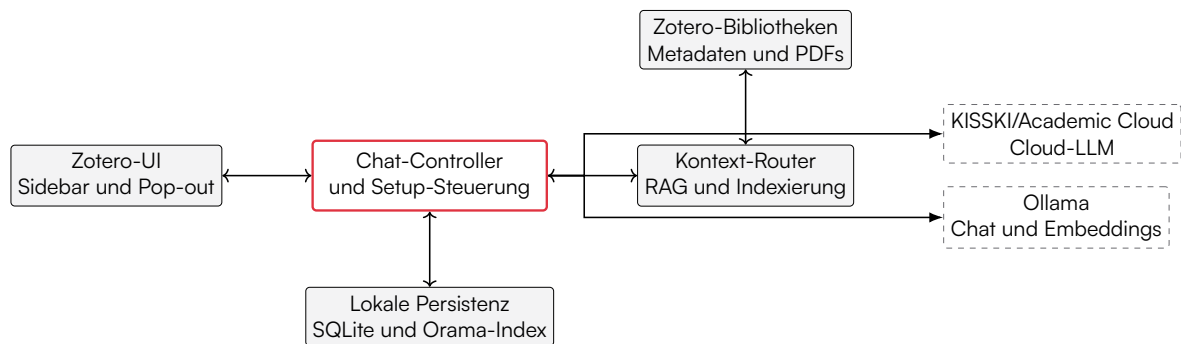


Abbildung 1: Vereinfachte Systemarchitektur von ZAIA

3.2 Zentrale Komponenten

Komponente	Verantwortung und Beispiele
UI-Schicht	Aufbau und Synchronisierung von Sidebar, Pop-out, Einstellungsseite, Modellfenster und Index-Manager
Chat-Controller	Chat-Lebenszyklus, Request-Aufbau, Streaming-Anzeige, Abbruch, Kontextsteuerung und Persistenzaufrufe
Kontext-Router	Entscheidung, ob keine Quellen, nur Metadaten, einzelne, gefilterte oder alle Paper benötigt werden
Paper-Pipeline	Item-Auflösung, PDF-Extraktion, Chunking, Metadatenformatierung und Retrieval
Indexierung	Hintergrundverarbeitung, Ereignisse, Abbruch und persistenter hybrider Orama-Index
Provider-Schicht	Einheitlicher Zugriff auf KISSKI und Ollama sowie getrennte lokale Embedding-Anbindung
Persistenz	SQLite-Repository für Chats und Nachrichten; Zotero-Präferenzen für Konfiguration; JSON-basierter Vektorindex
Betriebsschicht	Readiness-Prüfung, signaturgeprüftes Setup, Modell-Downloads und Ollama-Prozessverwaltung

3.3 Ablauf einer Anfrage

1. Der Chat-Controller validiert Eingabe und Betriebsbereitschaft des gewählten Providers sowie — falls aktiviert — des Embedding-Dienstes.
2. ZAIA persistiert den Nutzer-Prompt und ermittelt automatisch markierte sowie manuell angehängte Paper.
3. Wenn kein Kontext erzwungen wurde, bewertet der Kontext-Router die Anfrage anhand von Prompt, verfügbaren Metadaten und Bibliotheksbestand.
4. Die Paper-Pipeline erzeugt je nach Route einen Metadatenblock oder sucht relevante Chunks in bestimmten beziehungsweise allen indexierten Papern.
5. Der Chat-Controller kombiniert Systemkontext und bisherigen Verlauf zu einer Provider-unabhängigen Nachrichtenliste.

6. Der aktive Provider verarbeitet die Anfrage. Die Oberfläche zeigt den Fortschritt, übernimmt Antwort und Tokeninformationen und reagiert auf einen Abbruch durch die Nutzer:innen.
7. Nach erfolgreichem Abschluss werden Antwort und aktualisierte Chat-Metadaten lokal gespeichert und in Sidebar sowie Pop-out synchronisiert.

Fällt die semantische Suche aus, versucht ZAIA die Retrieval-Anfrage als Volltext-/Keyword-Suche weiterzuführen. Scheitert die Router-Entscheidung, greift ein älterer, einfacherer Kontextpfad. Diese Fallbacks verbessern die Fehlertoleranz, können aber eine schwächere Kontextauswahl bewirken.

3.4 Architekturentscheidungen und Abwägungen

Direkte Zotero-Integration. Die Erweiterung kann Auswahlzustand, Metadaten, Anhänge und Gruppenbibliotheken ohne Export nutzen. Dafür besteht eine enge Bindung an Zotero-APIs und an die unterstützte Hauptversion 9.

Provider-Abstraktion. Cloud und lokale Modelle folgen derselben Schnittstelle. Dadurch bleiben Chat- und Kontextlogik austauschbar. Unterschiede in Modellfähigkeiten, Antwortformaten und Tokenzählung müssen dennoch durch die Provider normalisiert werden.

Lokales RAG. Embeddings, Vektorindex und Chatdaten liegen standardmäßig lokal. Das reduziert Datenabfluss und laufende API-Kosten, verlangt aber lokale Rechenleistung, Speicherplatz und einen zusätzlichen Ollama-Dienst.

Explizite Indexkontrolle. Ein eigener Index-Manager macht Zustand und Fortschritt sichtbar und erlaubt gezielte Aktionen. Dies erhöht die Nachvollziehbarkeit, bringt gegenüber einer vollständig unsichtbaren Hintergrundindexierung aber zusätzliche Bedienkomplexität mit sich.

3.5 Datenhaltung und Datenschutz

Im reinen lokalen Modus werden Chat- und Embedding-Anfragen an die konfigurierte Ollama-Instanz gesendet; bei der Standardadresse bleibt die Verarbeitung auf dem eigenen Rechner. Chatverläufe liegen in `zaia-chats.sqlite`, der Rechercheindex in `orama_vector_index_v2.json` innerhalb des lokalen Zotero-Umfelds. API-Schlüssel werden als Zotero-Präferenz verwaltet und nicht in den Chatdatensätzen abgelegt.

Im Cloud-Modus werden der Prompt und die für die Antwort aufgebauten Nachrichten an den KISSKI-Endpunkt übertragen. Die Übermittlung von Paper-Kontext kann in den Einstellungen deaktiviert werden. Diese Option verhindert jedoch nicht die Cloud-Verarbeitung des eigentlichen Prompts. Für vertrauliche oder noch nicht veröffentlichte Inhalte ist daher der lokale Modus beziehungsweise eine vorab geklärte institutionelle Cloud-Nutzung vorzuziehen.

4 Projektvorgehen und Qualitätssicherung

4.1 Arbeitsteilung und iteratives Vorgehen

Der im Plugin selbst dokumentierte Teamzuschnitt verbindet klare Schwerpunkte mit gemeinsamer Umsetzung:

Teammitglied	Schwerpunkt
Frank Baum	Setup und KI-Anbindung
Vincent Korsch	UI-Design und Branding
Nicolas Medda	Embedding und Indexierung
Timo Rausch	KI-Anbindung und Architektur
Henrik Schneider	Indexierung und Datenspeicherung

Die Repository-Historie dokumentiert eine iterative Entwicklung vom ersten Commit am 11. Mai 2026 bis zum betrachteten Stand am 15. Juli 2026. Insgesamt umfasst sie 208 Commits, darunter 61 Merge-Commits. Feature-Branches und Pull Requests wurden unter anderem für UI, Chat-Persistenz, lokale Modelle, Text-Extraktion, Embeddings, Kontext-Routing, Einstellungen, Shortcuts und den Index-Manager eingesetzt. Der Tag `V.1.0.0` markiert den ersten abgeschlossenen Release-Stand.

4.2 Automatisierte Prüfungen

Zum Berichtsstand wurden alle vorhandenen Prüfungen erneut ausgeführt. Das Ergebnis waren 74 bestandene Tests in 14 Testdateien. Zusätzlich lief der Produktions-Build einschließlich TypeScript-Typprüfung ohne Fehler durch.

Prüfbereich	Abgedeckte Aspekte
Provider und HTTP	Basisklassen, Provider-Manager, KISSKI/Ollama, Fehlernormalisierung und HTTP-Client
Setup und Betrieb	Readiness-Logik, Ollama-Lebenszyklus, Installationsskripte und lokale Modellauswahl
Kontext und Retrieval	Embedding-Fallback, Kontext-Router, Metadatenfelder und Paper-Kontext-Pipeline
Index-Manager	Auswahlzustand, UI-Aktionen, Abbruch und Indexsteuerung
Integration	Provider-Pipeline sowie Paper-Kontext vom Zotero-Mock bis zum Retrieval-Ergebnis
Build	Add-on-Bündelung, XPI-Erzeugung und <code>tsc --noEmit</code>

4.3 Verbleibende Qualitätsgrenzen

Die Tests konzentrieren sich auf isolierte Logik und simulierte Integrationspfade. Im Repository ist keine automatisierte Ende-zu-Ende-Prüfung in einer real gestarteten Zotero-Instanz dokumentiert. Ebenso fehlen bislang eine systematische Usability-Studie, Lasttests mit sehr großen Bibliotheken und eine vergleichende Qualitätsmessung der Antworten verschiedener Modelle. Da PDF-Qualität, OCR, Hardware und Modellwahl stark variieren, müssen diese Punkte vor einem breiten produktiven Einsatz gesondert untersucht werden.

5 Ökonomische Betrachtung und Wettbewerbsanalyse

5.1 Zielgruppen und Nutzenversprechen

Zielgruppe	Nutzenversprechen
Studierende	Schnellere Orientierung in Seminar-, Abschluss- und Prüfungsliteratur ohne Wechsel aus Zotero
Forschende	Fragen, Vergleiche und Zusammenfassungen über kuratierte eigene Bestände mit wählbarem Datenschutzprofil
Arbeitsgruppen	Einbezug gemeinsamer Zotero-Bibliotheken und konsistenter Metadaten in Rechercheprozesse
Hochschulen	Offen lizenzierbare, lokal betreibbare Ergänzung zu institutionellen Cloud-Angeboten wie KISSKI

Der wichtigste qualitative Nutzen liegt in der Vermeidung von Medienbrüchen. Quellen müssen nicht erneut hochgeladen oder außerhalb Zotero organisiert werden. Der Kontext kann aus Auswahl, Metadaten und Bibliotheksindex erzeugt werden; das verkürzt insbesondere wiederkehrende Orientierungs- und Vergleichsaufgaben. Eine monetäre Bewertung wäre zum aktuellen Stand jedoch spekulativ, weil weder Bearbeitungszeiten noch Antwortqualität in einer kontrollierten Nutzerstudie gemessen wurden.

5.2 Aufwand und Kostenstruktur

Die Software selbst ist als Open-Source-Projekt ohne vorgesehene Lizenzgebühr angelegt. Kosten entstehen an anderen Stellen:

- **Entwicklung und Wartung:** Anpassungen an neue Zotero-Versionen, Modell-APIs, Betriebssysteme und Sicherheitsanforderungen erfordern kontinuierliche Pflege.
- **Cloud-Nutzung:** Für den KISSKI-Modus werden ein berechtigter Zugang und ein API-Schlüssel benötigt. Kosten beziehungsweise Kontingente hängen vom institutionellen Angebot ab.
- **Lokaler Betrieb:** Ollama vermeidet nutzungsabhängige API-Gebühren, beansprucht aber Arbeitsspeicher, Rechenzeit, Energie und Speicherplatz für Modelle und Index.
- **Support:** Installation, Modellwahl, OCR-Probleme und Datenschutzfragen erzeugen Unterstützungsbedarf, besonders bei einer hochschulweiten Bereitstellung.

Für eine Weiterführung bietet sich ein gemeinschaftlich gepflegtes Open-Source-Modell mit institutionellem Sponsoring an. Eine Hochschule könnte ein geprüftes Release, vorkonfigurierte KISSKI-Zugänge und Support bereitstellen, während die lokale Option Forschenden mit erhöhtem Schutzbedarf erhalten bleibt.

5.3 Wettbewerbsanalyse

Die folgende Einordnung verwendet öffentlich dokumentierte Funktionen mit Stand Juli 2026. „Nicht dokumentiert“ bedeutet nicht zwingend, dass eine Funktion technisch unmöglich ist, sondern dass sie auf den geprüften offiziellen Produktseiten nicht als regulärer Betriebsmodus beschrieben wird.

Angebot	Zotero-Bezug	Lokale KI	Eigener Quellenkontext	Zugang und Kosten
ZAIA	Native Erweiterung	Ollama für Chat und Embeddings	Persönliche und Gruppenbibliotheken, Metadaten, PDFs und lokaler Index	AGPL; lokale Infrastruktur bzw. KISSKI-Zugang erforderlich
PapersGPT	Native Zotero-Erweiterung	Lokale Modelle und Ollama dokumentiert	Einzelne/mehrere PDFs bis zur Bibliothek	Kostenlose Teststufe und bezahlte Stufen; Quellcode AGPL
Elicit	Import aus Zotero in Elicit Library	Nicht dokumentiert	Importierte Referenzen/PDFs, Suche, Extraktion und Reviews	Kostenloser Basic-Plan, kostenpflichtige Plus-/Pro-/Teamstufen
SciSpace	Zotero-Konto wird mit Webplattform verbunden	Nicht dokumentiert	Importierte PDFs und Copilot-/Library-Workflows	Kostenloser Basiszugang, kreditbasierte Abonnements

Geprüfte Produktquellen:

- PapersGPT: [GitHub-Repository](#) und [Preisseite](#)
- Elicit: [Library-Funktionen](#) und [Preisseite](#)
- SciSpace: [Anleitung zur Zotero-Verbindung](#) und [Tarifübersicht](#)

ZAIA differenziert sich gegenüber den beiden Webplattformen vor allem durch den Verbleib im bestehenden Zotero-Arbeitsablauf und den lokalen Retrieval-Stack. Nutzer:innen können sowohl Chatmodell als auch Embeddings lokal ausführen, ohne ihre Bibliothek zunächst in einen separaten Dienst zu importieren. Gegenüber PapersGPT ist diese Positionierung weniger eindeutig, weil auch dort native Zotero-Integration, lokale Modelle und bibliotheksweite Auswertung dokumentiert sind. ZAIA kann hier insbesondere mit der Anbindung an den deutschen Hochschulkontext, dem expliziten KISSKI-Pfad, der Unterstützung von Gruppenbibliotheken und einem transparent steuerbaren Index argumentieren.

Die etablierten Alternativen besitzen zugleich Vorteile: Elicit und SciSpace bieten breitere, spezialisierte Web-Workflows etwa für systematische Reviews und strukturierte Extraktion. PapersGPT dokumentiert eine größere Modellauswahl und weitergehende Batch-Funktionen. ZAIA ist demgegenüber ein junges Projekt mit kleinerem Wartungs- und Supportnetz. Seine attraktivste Zielgruppe sind daher Zotero-Nutzer:innen an Hochschulen, die einen nachvollziehbaren, lokal betreibbaren Assistenten mit institutioneller Cloud-Option suchen.

6 Fazit und Ausblick

ZAIA erreicht das zentrale Projektziel: Version 1.0.0 verbindet eine ausgereifte Zotero-Oberfläche mit lokaler und cloudbasierter KI, bibliotheksbezogener Kontextauswahl, semantischem Retrieval und lokaler Chat-Persistenz. Besonders relevant sind die durchgängige Integration in den Literaturbestand, die Unterstützung von Gruppenbibliotheken sowie die Wahl zwischen Komfort und lokalem Datenschutz. Der erfolgreiche Build und 74 bestandene automatisierte Tests belegen, dass nicht nur ein Oberflächenentwurf, sondern ein technisch zusammenhängender Prototyp vorliegt.

Gleichzeitig bleibt ZAIA ein Forschungs- und Lehrprojekt. Die Aussagequalität hängt von Modell, Prompt, Index und PDF-Text ab; KI-Antworten ersetzen keine Prüfung der Originalquellen. Für eine nachhaltige Weiterführung sollten die nächsten Schritte in folgender Reihenfolge bearbeitet werden:

1. Ende-zu-Ende-Tests in realen Zotero-Installationen und eine strukturierte Usability-Studie mit den primären Zielgruppen,
2. nachvollziehbare Quellenverweise aus Antworten zurück zu Paper, Chunk und Seitenzahl sowie Evaluation der Retrieval-Qualität,
3. Performance- und Robustheitstests mit großen, heterogenen Bibliotheken einschließlich OCR-fehlerhafter Dokumente,
4. automatisierte Release- und Updateprozesse, vollständige Installationsdokumentation und Prüfung weiterer Plattformen sowie
5. ein geklärtes Wartungs-, Datenschutz- und Supportkonzept für eine mögliche hochschulweite Bereitstellung.

Damit bietet das Projekt eine tragfähige Grundlage für weitere Entwicklung. Der größte nächste Erkenntnisgewinn liegt nicht in zusätzlichen Einzel-Features, sondern in der systematischen Prüfung, wie zuverlässig, verständlich und zeitsparend ZAIA im tatsächlichen wissenschaftlichen Arbeitsprozess ist.